

# Wreath Blueprint Generator App - Technical Specification Document

## 1. Executive Summary

### 1.1 Project Overview

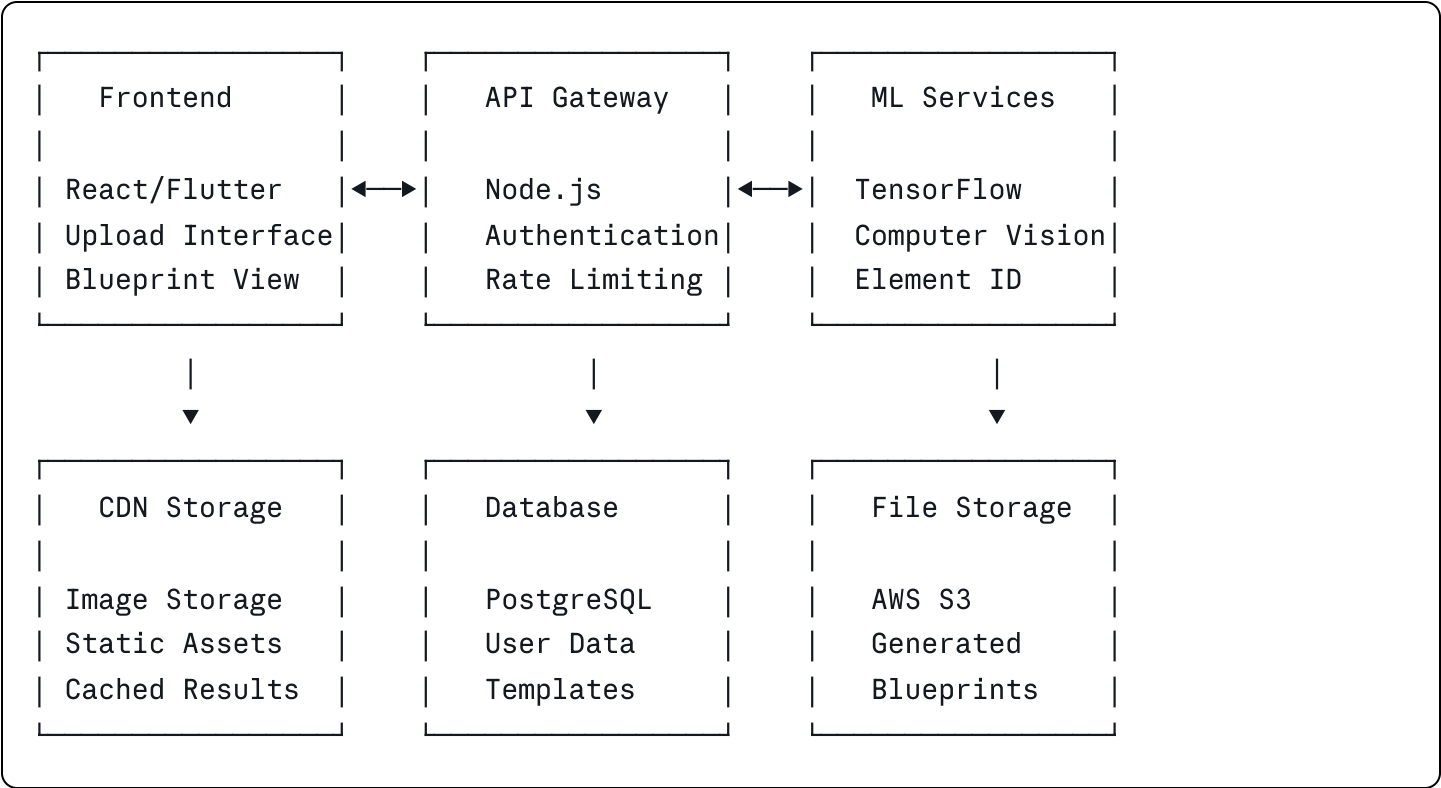
The Wreath Blueprint Generator is a comprehensive mobile and web application that uses computer vision and AI to analyze uploaded wreath images, identify floral elements, and generate precise technical blueprints with exact coordinates for recreation or commercial production.

### 1.2 Key Value Propositions

- Automated Analysis:** AI-powered identification of wreath components and dimensions
- Precise Blueprints:** Mathematical coordinate generation for exact reproduction
- Material Intelligence:** Comprehensive database of floral elements with sourcing information
- Export Flexibility:** Multiple output formats (PDF, SVG, CAD, shopping lists)

## 2. System Architecture

### 2.1 High-Level Architecture



## 2.2 Technology Stack

### Frontend

- **Mobile:** React Native (iOS/Android) or Flutter
- **Web:** React.js with TypeScript
- **UI Framework:** Material-UI or Tailwind CSS
- **State Management:** Redux Toolkit or Zustand
- **Image Handling:** React-Image-Crop, Fabric.js for annotations

### Backend

- **API Server:** Node.js with Express.js or Python with FastAPI
- **Authentication:** JWT with refresh tokens, OAuth2 integration
- **File Upload:** Multer (Node.js) or similar with virus scanning
- **Queue System:** Redis Bull Queue for processing jobs

### Database

- **Primary:** PostgreSQL for structured data
- **Cache:** Redis for session management and temporary data
- **Search:** Elasticsearch for element and template search

### AI/ML Pipeline

- **Computer Vision:** TensorFlow.js (client-side) + TensorFlow Serving (server-side)
- **Image Processing:** OpenCV, PIL/Pillow
- **Pre-trained Models:** YOLO v8 for object detection, Mask R-CNN for segmentation
- **Custom Models:** Fine-tuned on floral element dataset

### Infrastructure

- **Cloud Platform:** AWS or Google Cloud Platform
- **Containerization:** Docker with Kubernetes orchestration
- **CI/CD:** GitHub Actions or GitLab CI
- **Monitoring:** DataDog, New Relic, or Grafana

## 3. Core Functionality Specifications

### 3.1 Image Upload & Processing

#### 3.1.1 Upload Requirements

- **Supported Formats:** JPEG, PNG, WebP, HEIC
- **File Size Limits:** 50MB maximum, 1MB minimum recommended
- **Resolution Requirements:** Minimum 800x600, Recommended 2000x2000+
- **Quality Standards:** Automatic compression with quality preservation

#### 3.1.2 Preprocessing Pipeline

python

```
def preprocess_image(image_path):  
    # 1. Load and validate image  
    image = cv2.imread(image_path)  
  
    # 2. Noise reduction  
    denoised = cv2.bilateralFilter(image, 9, 75, 75)  
  
    # 3. Contrast enhancement  
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))  
    enhanced = clahe.apply(cv2.cvtColor(denoised, cv2.COLOR_BGR2GRAY))  
  
    # 4. Color space conversion  
    hsv = cv2.cvtColor(denoised, cv2.COLOR_BGR2HSV)  
  
    # 5. Edge detection for shape analysis  
    edges = cv2.Canny(enhanced, 50, 150)  
  
    return {  
        'original': image,  
        'enhanced': enhanced,  
        'hsv': hsv,  
        'edges': edges  
    }
```

### 3.2 Wreath Detection Algorithm

#### 3.2.1 Circle Detection

- **Method:** Hough Circle Transform with adaptive parameters
- **Fallback:** Ellipse detection for non-circular wreaths
- **Manual Override:** User can manually define wreath boundaries

### 3.2.2 Size Calibration Methods

1. **Reference Object Detection:** Automatic coin/ruler recognition
2. **Manual Input:** User provides actual diameter
3. **Common Object Recognition:** Detect hands, standard decorations
4. **Template Matching:** Compare with known wreath sizes

## 3.3 Element Identification System

### 3.3.1 Classification Categories

typescript

```
interface FloralElement {
  id: string;
  category: 'flower' | 'leaf' | 'berry' | 'branch' | 'decoration' | 'bow';
  subcategory: string; // rose, pine, holly, etc.
  color: {
    primary: string;
    secondary?: string;
    hex: string[];
  };
  position: {
    x: number;
    y: number;
    angle: number;
  };
  dimensions: {
    width: number;
    height: number;
    area: number;
  };
  confidence: number; // 0-1
  boundingBox: {
    x1: number;
    y1: number;
    x2: number;
    y2: number;
  };
}
```

### 3.3.2 ML Model Architecture

- **Base Model:** YOLOv8 for real-time object detection
- **Custom Classes:** 50+ floral element categories
- **Training Data:** 100,000+ annotated wreath images
- **Accuracy Target:** 85%+ for common elements

## 3.4 Coordinate Generation Engine

### 3.4.1 Mathematical Foundation

javascript

```

class CoordinateGenerator {
  constructor(centerX, centerY, outerRadius, innerRadius) {
    this.center = { x: centerX, y: centerY };
    this.outerRadius = outerRadius;
    this.innerRadius = innerRadius;
  }

  generateCircularCoordinates(numPoints) {
    const angleStep = 360 / numPoints;
    const coordinates = [];

    for (let i = 0; i < numPoints; i++) {
      const angle = i * angleStep;
      const radians = (angle * Math.PI) / 180;

      const x = this.center.x + this.outerRadius * Math.cos(radians);
      const y = this.center.y + this.outerRadius * Math.sin(radians);

      coordinates.push({
        index: i,
        angle: angle,
        x: Math.round(x * 100) / 100, // 2 decimal precision
        y: Math.round(y * 100) / 100,
        position: this.getPositionName(angle)
      });
    }

    return coordinates;
  }

  getPositionName(angle) {
    const positions = [
      { range: [0, 22.5], name: 'Top' },
      { range: [22.5, 67.5], name: 'Top-Right' },
      { range: [67.5, 112.5], name: 'Right' },
      { range: [112.5, 157.5], name: 'Bottom-Right' },
      { range: [157.5, 202.5], name: 'Bottom' },
      { range: [202.5, 247.5], name: 'Bottom-Left' },
      { range: [247.5, 292.5], name: 'Left' },
      { range: [292.5, 337.5], name: 'Top-Left' },
      { range: [337.5, 360], name: 'Top' }
    ];
  }
}

```

```
    return positions.find(p => angle >= p.range[0] && angle < p.range[1]).name;  
  }  
}
```

## 4. Database Schema Design

### 4.1 Core Tables

sql

```
-- Users table
```

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  username VARCHAR(100) UNIQUE,  
  subscription_tier VARCHAR(50) DEFAULT 'free',  
  credits_remaining INTEGER DEFAULT 3,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Wreaths table
```

```
CREATE TABLE wreaths (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id),  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  image_url VARCHAR(500),  
  image_width INTEGER,  
  image_height INTEGER,  
  actual_diameter_cm DECIMAL(8,2),  
  detected_diameter_px INTEGER,  
  center_x DECIMAL(8,2),  
  center_y DECIMAL(8,2),  
  outer_radius DECIMAL(8,2),  
  inner_radius DECIMAL(8,2),  
  processing_status VARCHAR(50) DEFAULT 'pending',  
  confidence_score DECIMAL(3,2),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Elements table
```

```
CREATE TABLE elements (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  wreath_id UUID REFERENCES wreaths(id) ON DELETE CASCADE,  
  element_type VARCHAR(100) NOT NULL,  
  element_subtype VARCHAR(100),  
  color_primary VARCHAR(7), -- hex color  
  color_secondary VARCHAR(7),  
  position_x DECIMAL(8,2) NOT NULL,  
  position_y DECIMAL(8,2) NOT NULL,  
  angle_degrees DECIMAL(5,2),
```



```

width_px INTEGER,
height_px INTEGER,
area_px INTEGER,
confidence DECIMAL(3,2),
bounding_box JSONB,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Coordinates table
CREATE TABLE coordinates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wreath_id UUID REFERENCES wreaths(id) ON DELETE CASCADE,
    point_index INTEGER NOT NULL,
    angle_degrees DECIMAL(5,2) NOT NULL,
    x_coordinate DECIMAL(8,2) NOT NULL,
    y_coordinate DECIMAL(8,2) NOT NULL,
    position_name VARCHAR(50),
    assigned_element_id UUID REFERENCES elements(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Templates table
CREATE TABLE templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    category VARCHAR(100),
    season VARCHAR(50),
    difficulty_level INTEGER CHECK (difficulty_level >= 1 AND difficulty_level <= 5)
    coordinate_pattern JSONB NOT NULL,
    element_suggestions JSONB,
    material_list JSONB,
    estimated_cost DECIMAL(8,2),
    popularity_score INTEGER DEFAULT 0,
    is_public BOOLEAN DEFAULT true,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Exports table
CREATE TABLE exports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wreath_id UUID REFERENCES wreaths(id),
    user_id UUID REFERENCES users(id),
    export_type VARCHAR(50) NOT NULL, -- 'pdf', 'svg', 'json', 'shopping_list'

```

```
file_url VARCHAR(500),
download_count INTEGER DEFAULT 0,
expires_at TIMESTAMP,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

## 4.2 Indexing Strategy

sql

```
-- Performance indexes
CREATE INDEX idx_wreaths_user_id ON wreaths(user_id);
CREATE INDEX idx_wreaths_status ON wreaths(processing_status);
CREATE INDEX idx_elements_wreath_id ON elements(wreath_id);
CREATE INDEX idx_elements_type ON elements(element_type, element_subtype);
CREATE INDEX idx_coordinates_wreath_id ON coordinates(wreath_id);
CREATE INDEX idx_templates_category ON templates(category, season);
CREATE INDEX idx_exports_user_wreath ON exports(user_id, wreath_id);

-- GiST index for spatial queries
CREATE INDEX idx_elements_position ON elements USING gist(
    point(position_x, position_y)
);
```

## 5. API Specifications

### 5.1 Authentication Endpoints

typescript

```
// POST /api/auth/register
interface RegisterRequest {
  email: string;
  username: string;
  password: string;
}

interface AuthResponse {
  user: {
    id: string;
    email: string;
    username: string;
    subscription_tier: string;
    credits_remaining: number;
  };
  access_token: string;
  refresh_token: string;
  expires_in: number;
}

// POST /api/auth/login
interface LoginRequest {
  email: string;
  password: string;
}

// POST /api/auth/refresh
interface RefreshRequest {
  refresh_token: string;
}
```

## 5.2 Core API Endpoints

typescript

```
// POST /api/wreaths/upload
interface UploadRequest {
  image: File;
  actual_diameter?: number;
  reference_object?: 'coin' | 'ruler' | 'hand';
  title?: string;
  description?: string;
}

interface UploadResponse {
  wreath_id: string;
  processing_job_id: string;
  estimated_completion_time: number; // seconds
}

// GET /api/wreaths/{wreath_id}/status
interface ProcessingStatus {
  status: 'pending' | 'processing' | 'completed' | 'failed';
  progress_percentage: number;
  current_stage: string;
  estimated_remaining_time?: number;
  error_message?: string;
}

// GET /api/wreaths/{wreath_id}/results
interface WreathAnalysis {
  wreath: {
    id: string;
    dimensions: {
      diameter_cm: number;
      center: { x: number; y: number };
      outer_radius: number;
      inner_radius: number;
    };
    confidence_score: number;
  };
  elements: FloralElement[];
  coordinates: CoordinatePoint[];
  material_summary: MaterialSummary;
}

// POST /api/wreaths/{wreath_id}/export
interface ExportRequest {
```

```
format: 'pdf' | 'svg' | 'json' | 'shopping_list';
options?: {
  include_measurements?: boolean;
  include_materials?: boolean;
  coordinate_precision?: number;
  paper_size?: 'A4' | 'Letter' | 'A3';
};
}

interface ExportResponse {
  export_id: string;
  download_url: string;
  expires_at: string;
  file_size: number;
}
```

## 6. Security & Privacy

### 6.1 Data Security

- **Encryption:** AES-256 for data at rest, TLS 1.3 for data in transit
- **Authentication:** JWT with short expiration + refresh token rotation
- **Authorization:** Role-based access control (RBAC)
- **File Upload:** Virus scanning, file type validation, size limits

### 6.2 Privacy Considerations

- **Image Storage:** User images automatically deleted after 30 days unless saved
- **Data Retention:** User data retained per GDPR/CCPA requirements
- **Analytics:** Anonymized usage data only, opt-in for detailed analytics
- **Third-party:** No sharing of user images with external services

### 6.3 Rate Limiting

javascript

```
// Rate limiting configuration
const rateLimits = {
  free_tier: {
    uploads_per_day: 3,
    exports_per_day: 5,
    api_requests_per_minute: 10
  },
  premium_tier: {
    uploads_per_day: 50,
    exports_per_day: 100,
    api_requests_per_minute: 60
  },
  professional_tier: {
    uploads_per_day: 500,
    exports_per_day: 1000,
    api_requests_per_minute: 120
  }
};
```

## 7. Performance Requirements

### 7.1 Response Time Targets

- **Image Upload:** < 5 seconds for 10MB file
- **Wreath Analysis:** < 30 seconds for standard resolution
- **Coordinate Generation:** < 2 seconds
- **Blueprint Export:** < 10 seconds for PDF generation

### 7.2 Scalability Targets

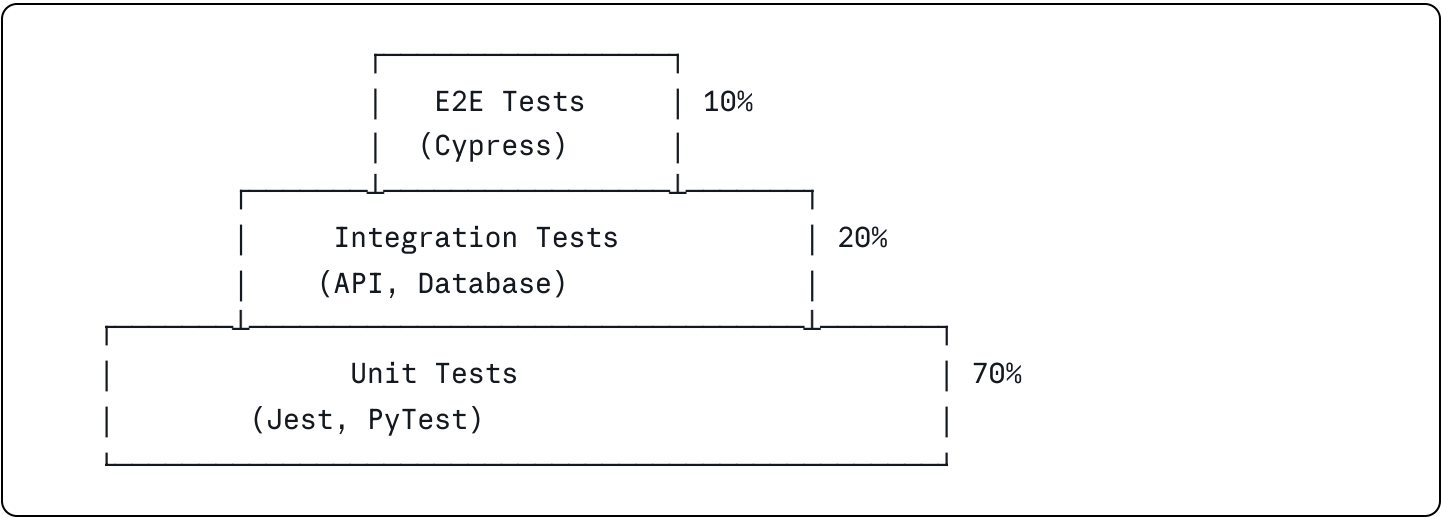
- **Concurrent Users:** 10,000 active users
- **Daily Uploads:** 50,000 image uploads
- **Storage Growth:** 1TB/month image storage
- **API Throughput:** 1,000 requests/second peak

### 7.3 Availability Requirements

- **Uptime:** 99.9% availability (8.77 hours downtime/year)
- **Disaster Recovery:** RTO 4 hours, RPO 1 hour
- **Geographic Distribution:** Multi-region deployment

## 8. Testing Strategy

### 8.1 Testing Pyramid



### 8.2 ML Model Testing

- **Accuracy Testing:** Confusion matrix, precision/recall metrics
- **Performance Testing:** Inference time benchmarks
- **Robustness Testing:** Adversarial examples, edge cases
- **A/B Testing:** Model version comparisons in production

## 9. Monitoring & Analytics

### 9.1 Application Metrics

- **Performance:** Response times, error rates, throughput
- **Business:** User engagement, conversion rates, retention
- **ML Metrics:** Model accuracy, prediction confidence, false positive rates
- **Infrastructure:** CPU, memory, disk usage, network latency

### 9.2 Alerting Strategy

```
yaml
```

```
alerts:
  critical:
    - error_rate > 5%
    - response_time_p99 > 10s
    - ml_model_accuracy < 80%
  warning:
    - error_rate > 2%
    - disk_usage > 80%
    - queue_depth > 100
```

## 10. Deployment & DevOps

### 10.1 Environment Strategy

- **Development:** Local Docker Compose setup
- **Staging:** Kubernetes cluster with production-like data
- **Production:** Multi-region Kubernetes deployment

### 10.2 CI/CD Pipeline

yaml



```
# GitHub Actions example
stages:
  - lint_and_test:
    - Code linting (ESLint, Black)
    - Unit tests (Jest, PyTest)
    - Security scanning (Snyk)

  - build_and_package:
    - Docker image build
    - Container security scan
    - Push to registry

  - deploy_staging:
    - Deploy to staging environment
    - Run integration tests
    - ML model validation

  - deploy_production:
    - Blue-green deployment
    - Health checks
    - Rollback capability
```

## 11. Future Enhancements

### 11.1 Phase 2 Features

- **3D Visualization:** Three.js integration for 3D wreath preview
- **AR Integration:** Mobile AR for wreath placement visualization
- **Social Features:** Community sharing, rating, commenting
- **AI Suggestions:** Intelligent element placement recommendations

### 11.2 Phase 3 Features

- **Video Analysis:** Process time-lapse creation videos
- **Seasonal Templates:** AI-generated seasonal design suggestions
- **E-commerce Integration:** Direct purchasing of identified materials
- **Professional Tools:** Batch processing, team collaboration features

## 12. Cost Estimation

### 12.1 Development Costs

- **Backend Development:** 6 months  $\times$  \$150k/month = \$900k
- **Frontend Development:** 4 months  $\times$  \$120k/month = \$480k
- **ML Development:** 8 months  $\times$  \$180k/month = \$1.44M
- **DevOps/Infrastructure:** 3 months  $\times$  \$100k/month = \$300k
- **Testing/QA:** 4 months  $\times$  \$80k/month = \$320k
- **Total Development:** \$3.44M

### 12.2 Operational Costs (Monthly)

- **Cloud Infrastructure:** \$15,000
- **ML Model Hosting:** \$8,000
- **Database:** \$3,000
- **CDN & Storage:** \$2,000
- **Third-party Services:** \$1,500
- **Monitoring:** \$500
- **Total Monthly:** \$30,000

### 12.3 Revenue Projections

- **Free Users:** 50% (ad-supported)
- **Premium (\$9.99/month):** 40%  $\times$  10,000 users = \$39,960/month
- **Professional (\$29.99/month):** 10%  $\times$  10,000 users = \$29,990/month
- **Total Revenue:** \$69,950/month (breakeven at ~4,300 users)

This technical specification provides a comprehensive foundation for building the Wreath Blueprint Generator app. The next steps would be to create detailed wireframes, set up the development environment, and begin with MVP development focusing on core image processing and coordinate generation functionality.